

Reinforcement Learning on Variable Impedance Controller for High-Precision Robotic Assembly

Jianlan Luo¹, Eugen Solowjow², Chengtao Wen², Juan Aparicio Ojea², Alice M. Agogino¹
Aviv Tamar³, Pieter Abbeel¹

Abstract—Precise robotic manipulation skills are desirable in many industrial settings, reinforcement learning (RL) methods hold the promise of acquiring these skills autonomously. In this paper, we explicitly consider incorporating operational space force/torque information into reinforcement learning; this is motivated by humans heuristically mapping perceived forces to control actions, which results in completing high-precision tasks in a fairly easy manner. Our approach combines RL with force/torque information by incorporating a proper operational space force controller; where we also exploit different ablations on processing this information. Moreover, we propose a neural network architecture that generalizes to reasonable variations of the environment. We evaluate our method on the open-source Siemens Robot Learning Challenge, which requires precise and delicate force-controlled behavior to assemble a tight-fit gear wheel set.

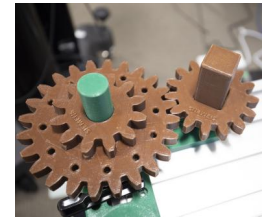
I. INTRODUCTION

Today, industrial robots deployed across various industries are mostly doing repetitive tasks. The overall task performance hinges on the accuracy of their controllers to track pre-defined trajectories. To this end, endowing these machines with a greater level of intelligence to autonomously acquire skills is desirable. The main challenge is to design adaptable, yet robust, control algorithms in the face of inherent difficulties in modeling all possible system behaviors and the necessity of behavior generalization. Reinforcement learning (RL) methods hold promises for solving such challenges, because they promise agents to learn behaviors through interaction with their surrounding environments and ideally generalize to new unseen scenarios [1, 2, 3, 4].

In this paper, we aim to learn policies that can assemble a high-precision gear set as shown in Fig.1. In real manufacturing, human labor can accomplish such high-accuracy complex tasks in a fairly easy manner. For example, a peg in hole insertion is achieved by “feeling” the contacts. This can be achieved with heuristics based on force feedback, for instance by probing the hole before inserting or moving the peg around the surface to search for the insertion point. However, designing robust strategies by properly processing observations is more desirable than heuristics or estimating perfect physical dynamics. RL allows to find control policies automatically for problems where traditionally heuristics have been used. The question arises *how do we properly integrate observed force information into reinforcement learning process to produce desirable behaviors?*



a) Robot learning for complex assemblies.



b) Assembled gear.

Fig. 1: Learning control policies for assembly tasks.

RL is a method for learning such reactive policies automatically, through trial and error interaction in the domain, guided only by a reward signal that specifies how well the robot is performing the task. In practice, RL requires an informative reward signal to work effectively, which can be hard to design automatically. With sparse reward that just specifies successful task completion, RL is prone to getting stuck in local optima. However operational space control could mitigate this problem by specifying high-level goals in task-space. [5, 6, 7, 8]. This corresponds to shaping the control actions so that policies only search the space where a “good” solution exists. We seek to answer the following three questions:

- 1) Will it help if control actions stem from an operational space controller? Since tool space forces provide the most straightforward and explicit information of such force-based tasks, can local trajectory optimizer with Markovian properties benefit from them?
- 2) Can adaptive impedance behavior be learned by our methods?
- 3) Can we learn a policy with generalization capabilities to local variations by explicitly considering force/torque measurements?

The contributions of this paper are answers to these questions. First, we find that local trajectory optimization can significantly benefit by using operation space controller for control actions. Second, we show adaptive compliant behavior can be acquired autonomously through interactions. Third, we propose a method to incorporate force/torque sensor data into global policy parameterized by neural networks.

II. PROBLEM STATEMENT AND RELATED WORK

a) *Problem Statement:* Consider the task of assembling the gear set shown in Fig. 1. The gear model was introduced by Siemens Corporation as a benchmark task for robotic

¹University of California, Berkeley, CA 94704, USA

²Siemens Corporate Technology, Berkeley, CA, 94704, USA

³Technion, Haifa, 3200003, Israel



a) Round peg in round hole. b) Gear wheel on shaft. c) Squared hole on squared shaft. d) Teeth Alignment.

Fig. 2: Four tasks that represent different assembly challenges. Each task requires a flexible control policy that needs to consider contacts and friction. Sub-figure a) to d) represents task 1 to 4 respectively.

assembly¹. The overall assembly task consists of four sequential steps, which are illustrated in Fig.2: first the robot needs to insert a cylindrical peg into its matching hole; then the large brown gear should be inserted through the gear shaft; then the small brown gear with the squared hole should be assembled; lastly the gear wheels need to be matched by aligning the corresponding gear teeth. In general the tolerances are tight. For example, step two requires tolerances tighter than 0.1 mm, which is beyond most deployed industrial robots' accuracy today. Additionally, in step two, the peg can freely rotate at contact, the gear must be precisely oriented to match the squared peg; in step three, the small brown gear must be rotated by the large brown gear properly so that they can align with each other. This poses additional challenges: since none of these pegs or gears are fixed during assembly, this added uncertainty makes assembly even more difficult.

b) *Related Work*: Recent advances in RL have gained great success in solving a variety of problems from playing video games [9, 10, 11] to robotic locomotion [12, 13, 14, 15, 16], manipulation [17, 18, 19, 20, 4, 21, 22, 23, 24, 25]. Reinforcement learning can be distinguished in model-based methods and model-free methods [1, 3, 2]. While model-based policy search is computationally more expensive than model-free methods, it requires less data to solve a task. Recent progress in the area of Deep Neural Networks suggests deploying them for parametrizing policies and other functions in RL methods [4, 26, 14]. This is often referred to as Deep Reinforcement Learning (DRL).

A recently developed model-based reinforcement learning algorithm called guided policy search (GPS) provided new insights into training end-to-end policy for solving contact-rich manipulation problems [15, 26, 27, 4, 28]; however, this method is not suitable for this high-precision setting because it has no means of avoiding local optima by its formulation. There are also approaches tackling this problem by explicitly modeling contact dynamics [29, 30, 31, 32, 33] Inoue et al. [34] use LSTM to learn two separate policies for finding and inserting a peg into a hole; however, their methods require several pre-defined heuristics, and also the action space is discrete.

Thomas et al. [35] combine RL with a motion planner to shape state cost in high-precision settings. This method

essentially learns a trajectory following torque controller, and assumes access to a trajectory planner that could roughly avoid local optima. They also encode such planned reference into a neural network with attention mechanism, they show good generalization results in simulation.

III. PRELIMINARIES

We consider all tasks here that can be described as moving already-grasped objects to their goal position. This is the most common setting in today's manufacturing. The success of such tasks can be measured as minimizing the distance between objects and their goal positions. We make no particular assumptions about encountered dynamics during tasks especially contacts. These need to be learned by the robot from various interaction with its environment. Let $\mathbf{x}_t$ and $\mathbf{u}_t$ denote robot states and actions respectively; $\ell(\mathbf{x}_t, \mathbf{u}_t)$ be the cost function related to the task, T be the time horizon of a task. Our problem can be formulated as

$$\begin{aligned} \min_{\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_T} \sum_{t=1}^T \ell(\mathbf{x}_t, \mathbf{u}_t) \\ \text{s.t. } \mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) \quad t = 1, 2, \dots, T-1, \end{aligned}$$

where f governs (unknown) system dynamics, $\mathbf{x}$, $\mathbf{u}$ can also be subject to other algebraic constraints.

We consider our control action $\mathbf{u}$ to be operational force controller $\mathcal{F}_{tip} = [F_x, F_y, F_z, M_x, M_y, M_z]$. They represent desired force/torque or impedance in operational space, our goal is to optimize them through reinforcement learning.

IV. REINFORCEMENT LEARNING WITH FORCE CONTROL

In this section, we first introduce operational space force control, and then move towards hybrid motion/force controller for more stable behaviors. We pick a particular model-based RL algorithm, iterative Linear-Quadratic-Gaussian (iLQG) for combining with these controllers, because it has been shown to be sample efficient. We then propose a neural network architecture that explicitly considers force information for better generalization purpose.

We explain our intuition for combining an operational force controller with RL using Fig.2(b). One successful strategy for inserting the gear with such high accuracy is to constrain the motion and forces somehow. For instance, if gear and the stand are in contact, we cannot move the gear

¹<http://www.usa.siemens.com/robot-learning>

downwards unless they are aligned. This is a natural constraint due to the rigid nature of environment. It is obvious that the task simplifies, if we constrain the motion to planar motions during the hole-searching phase; tilting would only help with fine adjustment when the gear is being inserted with high friction, this is an artificial constraint imposed by humans. Careful combinations of natural constraints and artificial constraints are essential to generate “task descriptions” in high-precision settings considered in this paper. Indeed, these can be regarded as Pfaffian constraints consisting of holonomic and nonholonomic components. Our method could be thought as implicitly generating such constraints.

Besides improving insertion accuracy we also learn a policy that is robust to local variations. We propose a neural network architecture as seen in Fig. 3, where force/torque measurements of the current time step are explicitly considered for control action derivation.

A. Operational Space Motion/Force Controller

Let $\mathcal{F}_{tip}$ be the desired wrench on the end-effector, we can then express the control law in joint space as [36]

$$M(q)\ddot{q} + c(q, \dot{q})\dot{q} + g(q) + J^T(q)\mathcal{F}_{tip} = \tau, \quad (1)$$

where q represents joint angles in generalized coordinates, $M(q)$ is the inertia, $c(q, \dot{q})$ is the Coriolis matrix, $g(q)$ are gravitational forces, $J(q)$ is the Jacobian, τ is the torque vector applied to manipulators' joints. In many force control tasks, robots move slowly, hence we ignore acceleration and velocity terms in Eq. 1. For a 7-DOF Sawyer manipulator arm that we consider in this paper, we can also project torques to its non-empty nullspace. Denoting the nullspace torque vector as τ_{null} , joint space control law is:

$$\tau = g(q) + J^T(q)\mathcal{F}_{tip} + [I - J^T(q)J^{T\dagger}(q)]\tau_{null}, \quad (2)$$

where $J^{T\dagger}(q)$ is the pseudo-inverse of $J^T(q)$. The control law in Eq.2 is appealing and simple, but it would generate undesirable and dangerous motion without enough resistance provided by the environment. In our experimental setup, we do not assume in-contact situations of objects being assembled, there is a relative open free-space that the robot needs to move through; directly applying Eq.2 would result in continuous acceleration. To mitigate this issue, in all our experiments, we wrap a position loop with small gains around the controller in Eq.2:

$$\begin{aligned} \tau = & \Sigma_1[K_{qp}(q - q^*) + K_{qd}(\dot{q} - \dot{q}^*)] + \Sigma_2 J^T(q)\mathcal{F}_{tip} \quad (3) \\ & + [I - J^T(q)J^{T\dagger}(q)]\tau_{null} + g(q), \quad (4) \end{aligned}$$

where K_{qp} and K_{qd} are diagonal gain matrices with small entries, q and $\dot{q}$ are current joint positions and velocities, q^* and $\dot{q}^*$ are the desired ones obtained via inverse kinematics from end-effector pose. Σ_1 and Σ_2 are diagonal matrices to weight motion and force portions, respectively. The resulting hybrid controller also achieves adaptive impedance behavior implicitly: $\mathcal{F}_{tip}$ is a time-varying linear-Gaussian controller conditional on robot configuration, which is detailed in Sec. IV-B. This learned piecewise linear controller will

ideally yield high-impedance when moving in free-space, and high-admittance whenever in contact; thus implicitly scaling motion-to-force ratio in aforementioned controller. Aforementioned $\mathcal{F}_{tip}$ will be calculated by an RL controller.

B. Iterative Linear-Quadratic-Gaussian Controller

The specific model-based reinforcement learning algorithm that we consider here is iterative Linear-Quadratic-Gaussian (iLQG). It is sample efficient and convenient second-order methods are available to solve it quickly [37]. Let $\omega = \{\mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{x}_T, \mathbf{u}_T\}$ denote a trajectory, such that $p(\omega) = p(\mathbf{x}_1) \prod_{t=1}^T p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)p(\mathbf{u}_t|\mathbf{x}_t)$, $\ell(\omega) = \sum_{t=1}^T \ell(\mathbf{x}_t, \mathbf{u}_t)$ denotes the cost along a single trajectory ω ; where $\mathbf{x}$ typically consists of joint angles, end-effector pose and their time derivatives. We wish to minimize this cost; the goal is to minimize the expectation $E_{p(\omega)}[\ell(\omega)]$ over trajectory ω by iteratively optimizing linear-Gaussian controllers and re-fitting linear-Gaussian dynamics. This algorithm iteratively linearizes the dynamics around the current nominal trajectory, constructs a quadratic approximation of the cost, computes the optimal actions with respect to this approximation of the dynamics and cost by dynamic programming, and forward runs resulting actions to obtain a new nominal trajectory. We adopt a slightly different version of iLQG proposed in [38, 15]. An additional entropy term is added into cost function such that $\tilde{\ell}(\mathbf{x}_t, \mathbf{u}_t) = \ell(\mathbf{x}_t, \mathbf{u}_t) - \mathcal{H}(p(\mathbf{u}_t|\mathbf{x}_t))$ to encourage exploration. It can be shown that the optimal control law to this problem is $p(\mathbf{u}_t|\mathbf{x}_t) = \mathcal{N}(\mathbf{K}_t\mathbf{x}_t + \mathbf{k}_t, \mathbf{C}_t)$, and $\mathbf{C}_t = Q_{\mathbf{u}, \mathbf{u}t}^{-1}$, where Q is cost to go, $\mathbf{K}_t = -Q_{\mathbf{u}, \mathbf{u}t}^{-1}Q_{\mathbf{u}, \mathbf{x}t}$ and $\mathbf{k}_t = -Q_{\mathbf{u}, \mathbf{u}t}^{-1}Q_{\mathbf{u}t}$, subscripts denote ordered derivatives at time t [38].

C. Interpretation as Learning Pfaffian Constraints

Our method could be regarded as generating Pfaffian constraints for each task. At contact, we can formulate holonomic and nonholonomic constraints enforced by the rigid environment as Pfaffian constraints:

$$A(q)\mathcal{V} = 0, \quad (5)$$

where $\mathcal{V}$ is the operational space twist in $SE(3)$ that $\mathcal{V} \in \mathbb{R}^6$, $A(q) \in \mathbb{R}^{k \times 6}$, k is the number of natural constraints. In motion control part, we can write down operational space dynamics of the robot as:

$$\mathcal{F} = \Lambda(q)\dot{\mathcal{V}} + \eta(q, \mathcal{V}), \quad (6)$$

where $\Lambda(q) = J^{-T}(q)M(q)J^{-1}(q)$, $\eta(q, \mathcal{V}) = J^{-T}(q)c(q, J^{-1}\mathcal{V}) - \Lambda(q)J(q)J^{-1}(q)\mathcal{V}$. This is essentially the same motion dynamics without $\mathcal{F}_{tip}$ expressed in Eq.1, but calculated in operational space.

By combining Eq.5 and Eq.6, we can express constrained dynamics for this hybrid motion/force controller as

$$\mathcal{F} = \Lambda(q)\dot{\mathcal{V}} + \eta(q, \mathcal{V}) + \underbrace{A^T(q)\lambda}_{\mathcal{F}_{tip}}, \quad (7)$$

where $\lambda \in \mathbb{R}^k$; and in this case, requested wrenches $\mathcal{F}_{tip}$ must lie in the column space of $A^T(q)$. In general, constraints

$A(q)$ come from the environment the robot is interacting with. Abstracting this type of constraint for each individual manipulation task can be time-consuming and prone to errors. Instead, if we let $\mathcal{F}_{tip}$ be learned by RL through continuous interactions; we can roughly regard this process as iteratively improving $A(q)$ to generate increasingly accurate description of tasks, which is a key ingredient in high-precision settings.

D. Training Neural Network controller using MDGPS

We introduce a novel neural network architecture to process noisy force/torque readings from wrist sensors or other sources, which provide measurements in tool space. The neural network is shown in Fig. 3. Force/torque information is filtered with a low-pass filter (LFP), then concatenated in the second last layer of the neural network. Intuitively, we would like to provide most direct haptics information to the neural network as principle features; also avoiding the neural network to establish unreasonable correspondence between external force/torque readings and robot internal states. Also, the robot needs to move in free space before it is in contact; and the learned policy should be robot-configuration-dependent rather than force-dependent in free space. Since F/T readings are noisy, the learned policy dependent on robot state and coupled F/T generates random motions, if we directly treat F/T as an input to the first layer of the neural network.

For training this neural network, we adopt the mirror descent guided policy search (MDGPS) algorithm in [27]. Denote the neural network parameterized by θ as $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$, the goal is to minimize the KL-divergence between linear-Gaussian controller learned via iLQG:

$$\min \mathbb{D}_{KL}(\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)||p(\mathbf{u}_t|\mathbf{x}_t)) \quad \forall \mathbf{x}_t, \mathbf{u}_t, t,$$

this can be implemented by supervised learning. These guided policy search methods also have some mechanism to enforce agreement between distributions of local policy and global policy by adding an additional KL-divergence cost [27, 26, 4].

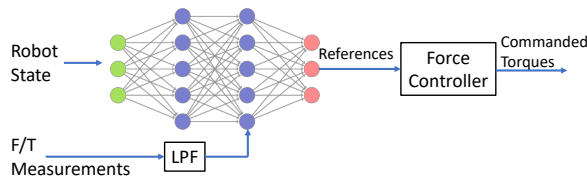


Fig. 3: Neural Network Architecture

We summarize our method in Alg.1.

V. EXPERIMENTS

In this section we answer the following questions. (1) How does the proposed iLQG with force control perform? Is it actively exploiting contact constraint dynamics as we hypothesized? How does it compare to its ablations where force information is integrated differently? (2) How does the

Algorithm 1 Force-based RL controllers

- 1: **for** iteration $k \in \{1, \dots, K\}$ **do**
- 2: Train local RL controller using iLQG, where $\mathbf{u}_t$ is set as operational space force controller
- 3: Project calculated operational control to joint torque using Eq.3
- 4: Train neural network controller using MDGPS[15]
- 5: **end for**

proposed neural network architecture improve local generalization? How does it compare to its ablations?

A. Experimental Setup Details

We evaluate our methods on four assembly tasks, which are shown in Fig.2. We use a Rethink Robotics Sawyer robot. Sawyer offers an interface to query its wrist force/torque measurement, the noise levels (estimated standard deviations) for F_x, F_y are 2.0 N; F_z is 0.5 N; M_x, M_y are 0.5 Nm; and M_z is 0.1 Nm. Sawyer is commanded via ROS at 20 Hz. During training, we take four roll-outs per iteration. Typically, it takes three iterations to achieve successful behaviors, five iterations for convergence. We define a plane by three points in end-effector space, the cost function is a weighted mixture of the ℓ_1 and ℓ_2 norms of the differences between the current plane and the target plane as specified by the three aforementioned points.

B. Assembly Performance Results

We compare our method of Sec. IV-B with the following baselines:

- **Kinematics Only:** For task 1 and task 2, we only specify target poses; for the more difficult task 3 and task 4, we also introduce several way-points. Note that for task 1 and 2, we should get the same result every single time since robot kinematic controller is deterministic as well as these tasks. But for task 3 and 2, the peg and gear can move freely, so it is hard to specify the desired trajectory.
- **iLQG with torque control:** This is the main baseline for comparison. The control actions from iLQG are directly the seven joint torques. For comparison, we use the same cost function as in our method, i.e., sparsely-defined target end-effector pose, no intermediate way-points are introduced.
- **iLQG with torque control, augmented state space:** We augment the state space with the F/T vector such that $\tilde{\mathbf{x}}_t = [\mathbf{x}_t, \mathbf{f}_t]$, where $\mathbf{f}_t$ are F/T measurements. We apply direct torque control. The purpose of this is to verify if other formulation other than what we proposed could also actively use this additional information.
- **Our Method:** We refer to the operational space controller in Section IV with iLQG.
- **Our Method with augmented state space:** Additional to operational space controller, we augment the state space to $\tilde{\mathbf{x}}_t = [\mathbf{x}_t, \mathbf{f}_t]$. This experiment is for verifying if our method can be further improved.

A success is considered if an object is being assembled to a desired pose with defined tolerance. We report success rates for each individual task separately, because we train an individual policy for each task. However, it would be straightforward to report overall success rate by multiplying individual success rates together since policies are trained independently. We execute learned policies after training to calculate the success rates. Table I presents aforementioned success rates for four different tasks.

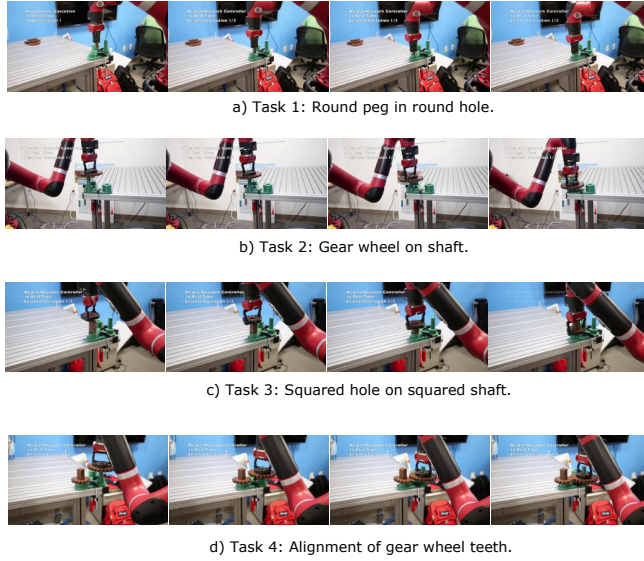


Fig. 4: Snapshots of experimental runs for the four studied assembly tasks.

TABLE I: Comparison of success rates for different tasks. Baseline 1 refers to kinematics only; baseline 2 refers to iLQG with direct torque control; baseline 3 refers to iLQG with direct torque control, augmented state space, our method w/ augmented refers augmented state space in our method.

	Task 1	Task 2	Task 3	Task 4
baseline 1	0/5	0/5	0/5	0/5
baseline 2	1/5	0/5	0/5	0/5
baseline 3	0/5	0/5	0/5	0/5
our method	5/5	5/5	2/5	4/5
our method w/ augmented	5/5	5/5	3/5	3/5

We interpret these results several fold: (1) kinematics baseline fails consecutively, this confirms the required accuracy and complexity for the gear set; (2) an iLQG with torque control, but without extensive cost shaping fails; the single success we observed is due to Gaussian noise in the controller, which generated some lucky motion to insert, and it is on the easiest task. (3) we did not find reliable improvement by augmenting state space with F/T information. Since F/T signals are not Markovian, fitting a time-correlated dynamics model to them does not produce meaningful information.

We made several interesting observations during the experiments. During task one, the robot moves quickly in free-space to reach in-contact status, then it reduces its speed to slowly probe around, trying to "feel" the surface; once it has a level of confidence of the hole's position, it becomes aggressive towards the goal it predicted, resulting in quick motions followed by a large downward force to complete insertion. The most interesting experiment is task 3, where the added uncertainty comes from a rotating peg. The robot first brings the small gear in contact with the peg, while applying a downward force so that small gear would not fall into free-space again; but this amount of downward force also allows room for applying additional rotating torque to the peg and gear aligning them with each other roughly; then the downward force increases to try insertion, if not successful, downward forces decrease but small horizontal force are also observed to fine-tune poses, this procedure iterates until the peg is fully inserted. This kind of behavior roughly aligns with humans' heuristics when facing such tasks. These behaviors can be found in the supplemental video².

Fig.5 shows computed actions (only desired forces in x-directions and y-directions are plotted) for task 2 during one successful insertion. It is interesting to observe how the variance computed by the policy changes over time. Initially, there is a certain level of variance for exploration to search for the target position; once the policy is confident about the goal, the variance reduces dramatically, the robot aggressively moves the object towards the goal; finally during the insertion phase, a certain level of noise is again injected for fine-tuning the gear's pose to overcome friction. This force-based insertion pattern is automatically discovered through interactions by the algorithm, and matches a human's intuition on such tasks well. Fig.6 presents F/T measurements during a successful insertion. We can observe peaks both in force and torque data, indicating some critical phases, e.g., contact. This motivates explicit use of F/T measurements, because of its informative nature.

C. Generalization Results

We only train the neural network controller based on a single instance of the local linear-Gaussian controller. Hence, our purpose is not to see if the trained neural network controller can interpolate between multiple local controllers as in original guided policy search methods [4]. Instead, we are interested in examining if the proposed architecture can effectively use F/T information and adapt to environment variations. We compare our proposed neural network against two baselines: one is to directly input F/T information into the first layer of the neural network; second is the iLQG controller that the neural network controller is trained on. We again only consider task 2 in these generalization experiments. We design our experiments as following: we train these three policies towards with the goal fixed, i.e., base does not move; after all policies are trained, we move

²<https://www.youtube.com/watch?v=kqAlt1PaT8E>

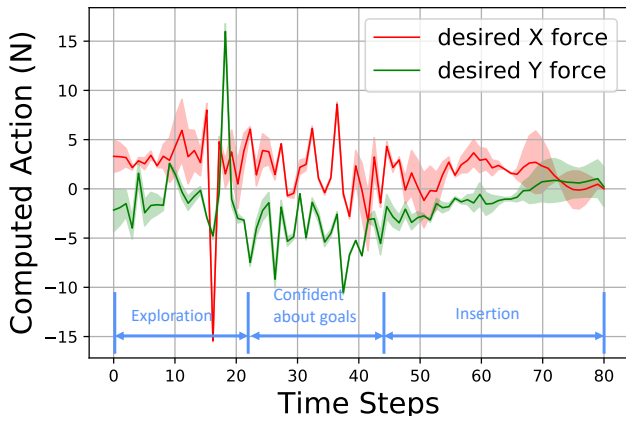


Fig. 5: Action computed by learned policy during one successful insertion. Solid line shows computed action mean, and error bar for computed variance.

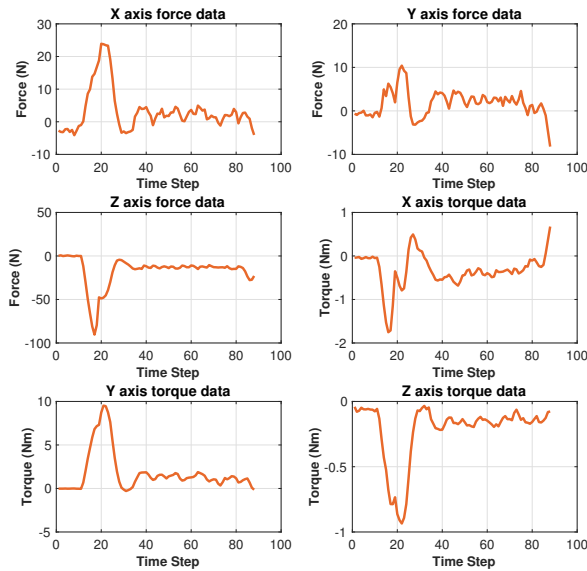


Fig. 6: Six degree of freedom force torque measurements from a successful Task 2 insertion.

base to slightly different positions but policies will be kept unchanged; then we count success rates for these variations of different base positions. The intuition is that even base is moved, the proposed neural network controller should still be able to find the hole if F/T information is actively used, since variations in base positions bear same force pattern in terms of peg-hole insertions. The comparison with iLQG baseline is due to the fact that slight variations in the goal position could also result in low cost in LQR. We want to distinguish between this and active F/T information.

We test these three methods in three different settings where the base is moved 1cm, 2cm, 5cm respectively. Success rates are reported in Table II

For neural networks that input F/T data to their first layer, the training was often aborted due to large KL-divergence

TABLE II: Comparison of success rate in generalization capability

	1cm	2cm	5cm
iLQG	8/10	5/10	6/10
our method	9/10	8/10	6/10
F/T input to first layer	0/10	0/10	0/10

between iLQG and neural network; for few scenarios we could successfully train a policy, it often generates random, undesirable motions in free-space before insertion; it behaved poorly on three experiments. This validates our hypothesis for not establishing correspondence between external F/T readings and robot internal states.

For the comparison of the proposed neural network controller against iLQG controller, we found that neural network controller produced slightly better results; but at the mean time, the variance of the result was also high. The neural network controller did not outperform iLQG consistently. Getting a more accurate six axes F/T controller might mitigate this issue.

VI. CONCLUSIONS AND FUTURE WORK

In this paper we combine RL with an operational space force controller to solve the problem of high-precision assembly. We show that RL essentially automates the generation of Pfaffian constraints in operational space constrained dynamics, which we regard as a crucial ingredient for high-precision tasks. We specifically exploited one of the model-based RL algorithm, iLQG, compared with several ablations, results show that our method performs best in this high-precision settings. We also introduced a neural network architecture that explicitly considers force/torque information in decision making process, which leads to a better result of generalization.

One future direction is to add raw vision and tactile inputs to the architecture in Figure 3, thus becoming an end-to-end multi-modal neural network. It would be interesting to see if such learned policy can succeed from arbitrary starting positions in free space, and tactile sensing could further improve policy performance. Another interesting direction is to explicitly model contact, and encode such information as priors for a more structured Pfaffian constraint matrix, this would further reduce sample complexity and serve as a general primitive for policy transfer.

ACKNOWLEDGEMENT

This work is partially funded by Siemens. Authors would like to thank Tobias Johannink for generous help on setting up the experiments.

REFERENCES

- [1] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. Vol. 1. MIT press Cambridge, 1998.
- [2] M. P. Deisenroth, G. Neumann, J. Peters, et al. "A survey on policy search for robotics". In: *Foundations and Trends in Robotics* 2.1–2 (2013), pp. 1–142.

- [3] J. Kober, J. A. Bagnell, and J. Peters. "Reinforcement learning in robotics: A survey". In: *The International Journal of Robotics Research* 32.11 (2013), pp. 1238–1274.
- [4] S. Levine, C. Finn, T. Darrell, and P. Abbeel. "End-to-end training of deep visuomotor policies". In: *The Journal of Machine Learning Research* 17.1 (2016), pp. 1334–1373.
- [5] O. Khatib. "A unified approach for motion and force control of robot manipulators: The operational space formulation". In: *IEEE Journal on Robotics and Automation* 3.1 (Feb. 1987), pp. 43–53.
- [6] J. Peters and S. Schaal. "Learning to Control in Operational Space". In: *The International Journal of Robotics Research* 27.2 (2008), pp. 197–212.
- [7] J. Nakanishi, R. Cory, M. Mistry, J. Peters, and S. Schaal. "Operational Space Control: A Theoretical and Empirical Comparison". In: *The International Journal of Robotics Research* 27.6 (2008), pp. 737–757.
- [8] J. Buchli, F. Stulp, E. Theodorou, and S. Schaal. "Learning variable impedance control". In: *The International Journal of Robotics Research* 30.7 (2011), pp. 820–833.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. "Playing atari with deep reinforcement learning". In: *arXiv preprint arXiv:1312.5602* (2013).
- [10] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. "Asynchronous methods for deep reinforcement learning". In: *International Conference on Machine Learning*. 2016, pp. 1928–1937.
- [11] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. "Continuous control with deep reinforcement learning". In: *arXiv preprint arXiv:1509.02971* (2015).
- [12] P. Abbeel, A. Coates, M. Quigley, and A. Y. Ng. "An Application of Reinforcement Learning to Aerobatic Helicopter Flight". In: *Proceedings of the 19th International Conference on Neural Information Processing Systems*. NIPS'06. Canada: MIT Press, 2006, pp. 1–8. URL: <http://dl.acm.org/citation.cfm?id=2976456.2976457>.
- [13] J. Luo, R. Edmunds, F. Rice, and M. Agogino. "Tensegrity Robot Locomotion under Limited Sensory Inputs via Deep Reinforcement Learning". In: *Robotics and Automation (ICRA), 2018 IEEE International Conference on*. IEEE. 2018.
- [14] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. "Trust region policy optimization". In: *International Conference on Machine Learning*. 2015, pp. 1889–1897.
- [15] S. Levine and P. Abbeel. "Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics". In: *Advances in Neural Information Processing Systems (NIPS)*. 2014.
- [16] T. Zhang, G. Kahn, S. Levine, and P. Abbeel. "Learning deep control policies for autonomous aerial vehicles with mpc-guided policy search". In: *Robotics and Automation (ICRA), 2016 IEEE International Conference on*. IEEE. 2016, pp. 528–535.
- [17] S. Levine, N. Wagener, and P. Abbeel. "Learning Contact-Rich Manipulation Skills with Guided Policy Search". In: *International Conference on Robotics and Automation (ICRA)*. 2015.
- [18] Y. Chebotar, M. Kalakrishnan, A. Yahya, A. Li, S. Schaal, and S. Levine. "Path integral guided policy search". In: *Robotics and Automation (ICRA), 2017 IEEE International Conference on*. IEEE. 2017, pp. 3381–3388.
- [19] J. Peters, K. Mülling, and Y. Altun. "Relative Entropy Policy Search." In: *AAAI*. Atlanta. 2010, pp. 1607–1612.
- [20] J. Fu, S. Levine, and P. Abbeel. "One-shot learning of manipulation skills with online dynamics adaptation and neural network priors". In: *Intelligent Robots and Systems (IROS), 2016 IEEE/RSJ International Conference on*. IEEE. 2016, pp. 4019–4026.
- [21] A. Tamar, G. Thomas, T. Zhang, S. Levine, and P. Abbeel. "Learning from the Hindsight Plan – Episodic MPC Improvement". In: *ArXiv e-prints* (Sept. 2016). arXiv: 1609.09001 [cs.RO].
- [22] P. Englert and M. Toussaint. "Learning manipulation skills from a single demonstration". In: *The International Journal of Robotics Research* 37.1 (2018), pp. 137–154.
- [23] I. Lenz, R. A. Knepper, and A. Saxena. "DeepMPC: Learning Deep Latent Features for Model Predictive Control". In: *Robotics: Science and Systems*. 2015.
- [24] J. Luo, E. Solowjow, C. Wen, J. A. Ojea, and A. M. Agogino. "Deep Reinforcement Learning for Robotic Assembly of Mixed Deformable and Rigid Objects". In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2018, pp. 2062–2069.
- [25] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine. "Residual Reinforcement Learning for Robot Control". In: *CoRR abs/1812.03201* (2018). arXiv: 1812.03201. URL: <http://arxiv.org/abs/1812.03201>.
- [26] S. Levine and V. Koltun. "Guided policy search". In: *International Conference on Machine Learning*. 2013, pp. 1–9.
- [27] W. H. Montgomery and S. Levine. "Guided policy search via approximate mirror descent". In: *Advances in Neural Information Processing Systems*. 2016, pp. 4008–4016.
- [28] Y. Chebotar, K. Hausman, M. Zhang, G. Sukhatme, S. Schaal, and S. Levine. "Combining Model-Based and Model-Free Updates for Trajectory-Centric Reinforcement Learning". In: *International Conference on Machine Learning (ICML) 2017*. Aug. 2017.
- [29] S. S. M. Salehian and A. Billard. "A Dynamical-System-Based Approach for Controlling Robotic Manipulators During Noncontact/Contact Transitions". In: *IEEE Robotics and Automation Letters* 3.4 (Oct. 2018), pp. 2738–2745.
- [30] S. M. Khansari-Zadeh and A. Billard. "Learning Stable Nonlinear Dynamical Systems With Gaussian Mixture Models". In: *IEEE Transactions on Robotics* 27.5 (Oct. 2011), pp. 943–957.
- [31] I. Mordatch, E. Todorov, and Z. Popović. "Discovery of Complex Behaviors Through Contact-invariant Optimization". In: *ACM Trans. Graph.* 31.4 (July 2012), 43:1–43:8.
- [32] I. Mordatch, K. Lowrey, and E. Todorov. "Ensemble-CIO: Full-body dynamic motion planning that transfers to physical humanoids". In: *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sept. 2015, pp. 5307–5314.
- [33] I. Mordatch, K. Lowrey, G. Andrew, Z. Popovic, and E. V. Todorov. "Interactive Control of Diverse Complex Characters with Neural Networks". In: *Advances in Neural Information Processing Systems* 28. Ed. by C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett. Curran Associates, Inc., 2015, pp. 3132–3140.
- [34] T. Inoue, G. De Magistris, A. Munawar, T. Yokoya, and R. Tachibana. "Deep reinforcement learning for high precision assembly tasks". In: *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on*. IEEE. 2017, pp. 819–825.
- [35] G. Thomas, M. Chien, A. Tamar, J. Aparicio Ojea, and P. Abbeel. "Learning Robotic Assembly from CAD". In: *ArXiv e-prints* (). arXiv: 1803.07635 [cs.RO].
- [36] K. M. Lynch and F. C. Park. *Modern Robotics: Mechanics, Planning, and Control*. 1st. New York, NY, USA: Cambridge University Press, 2017.
- [37] E. Todorov and W. Li. "A generalized iterative LQG method for locally-optimal feedback control of constrained nonlinear

- stochastic systems”. In: *Proceedings of the 2005, American Control Conference, 2005*. June 2005, 300–306 vol. 1.
- [38] B. D. Ziebart, J. A. Bagnell, and A. K. Dey. “Modeling Interaction via the Principle of Maximum Causal Entropy”. In: *Proceedings of the 27th International Conference on International Conference on Machine Learning. ICML’10*. Haifa, Israel: Omnipress, 2010, pp. 1255–1262.